

EduGenie — Google Gemini Powered Learning Assistant

Project Documentation

Project Type: AI-Powered Learning Platform for Students

Technology: React.js + Node.js/Express.js + MongoDB Atlas + Google Gemini API

Project Status: Fully implemented and tested locally

Deployment Status: Not publicly deployed

Project Description

EduGenie is a modern AI-powered learning platform designed for college students. It helps students understand concepts, ask doubts, generate structured notes, practice AI-generated quizzes, evaluate answers, create personalized study plans, and track learning progress.

EduGenie is designed as a structured learning platform rather than a simple chatbot. It combines authentication, feature-specific AI workflows, database persistence, learning history, quiz results, study planning, and progress tracking.

Core Capabilities

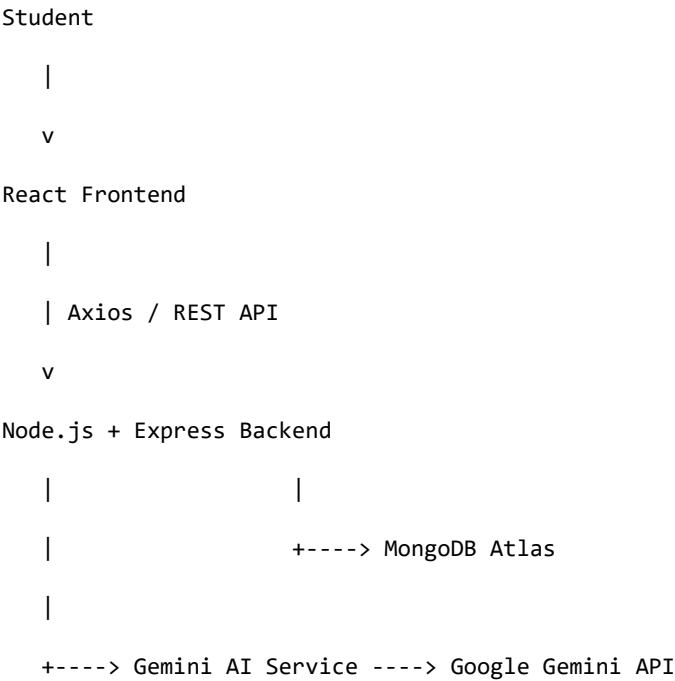
- Student registration and email/password login
- AI Tutor for doubt clarification and step-by-step learning
- Simple topic explanation with difficulty levels
- Structured study-notes generation
- AI quiz generation and quiz execution
- Answer evaluation with feedback
- Personalized study-plan generation
- Learning history and activity tracking
- Progress dashboard and learning statistics
- Profile, settings and responsive EdTech UI

1. Technical Architecture

EduGenie follows a modular full-stack architecture. React handles the user interface and routing, Node.js/Express provides secure REST APIs, MongoDB Atlas stores application data, and a dedicated Gemini service handles AI operations.

Layer	Technology	Responsibility
Frontend	React.js, JavaScript, HTML5, CSS3	Pages, components, forms and responsive UI
Routing	React Router	Public and protected navigation
HTTP Client	Axios	Frontend-to-backend API communication
Backend	Node.js + Express.js	REST APIs and business logic
Authentication	JWT + bcrypt	Authentication and password hashing
Database	MongoDB Atlas + Mongoose	Users, history, quizzes, plans and progress
AI	Google Gemini API	Educational generation and evaluation
Configuration	dotenv	Environment-variable management

High-Level Data Flow



The Gemini API key remains on the backend and is never exposed directly to the React client.

2. Prerequisites and Environment

- Node.js and npm
- React.js development environment
- JavaScript, HTML5 and CSS3
- React Router and Axios
- Node.js and Express.js
- MongoDB Atlas
- Google Gemini API access
- Git and GitHub
- VS Code and a modern browser

Environment Variables

```
GEMINI_API_KEY=your_gemini_api_key
```

```
MONGODB_URI=your_mongodb_connection_string
```

```
JWT_SECRET=your_jwt_secret
```

```
PORT=5000
```

The values above are placeholders. Real keys, database passwords and JWT secrets must remain private in the local .env file.

Git Ignore

```
.env
```

```
node_modules/
```

3. Project Workflow

Milestone 1: Requirements and Architecture

- Analyze requirements
- Define frontend/backend/database/AI responsibilities
- Plan reusable components and protected routes
- Define database models and API boundaries

Milestone 2: Project Setup

- Create frontend and backend structure
- Install dependencies
- Configure environment variables
- Connect MongoDB Atlas
- Configure JWT and bcrypt

Milestone 3: Authentication

- Registration
- Login
- Validation
- Password hashing
- JWT authentication
- Protected routes

Milestone 4: AI Learning Features

- Gemini service layer
- AI Tutor
- Topic explanation
- Notes
- Quiz
- Answer evaluation
- Study plan

Milestone 5: Learning Data

- Persist learning activity
- Store quiz results
- Store study plans
- Calculate progress from stored records

Milestone 6: UI/UX and Verification

- Responsive UI
- Loading/error/empty/success states
- API integration
- Build and runtime verification

4. Google Gemini API Integration

EduGenie uses Google Gemini as the AI engine for educational assistance. The backend contains a dedicated AI service layer so that the frontend never directly communicates with the Gemini API.

Secure Setup

1. Generate or obtain a Google Gemini API key.
2. Store it in the backend .env file.
3. Load it with dotenv.
4. Use it only from the backend Gemini service.
5. Keep .env in .gitignore.

Environment Variable

```
GEMINI_API_KEY=your_gemini_api_key
```

AI Service Responsibilities

- Generate explanations
- Generate notes
- Generate quizzes
- Evaluate answers
- Generate study plans
- Handle AI failures gracefully
- Use separate prompts for each feature

5. Database Architecture

MongoDB Atlas is used as the database and Mongoose provides schema-based database interaction.

Entity	Purpose	Example Data
User	Account/authentication	Name, email, hashed password
LearningHistory	Learning activity	Feature, topic, response, timestamp
QuizResult	Quiz attempts	Questions, answers, score, percentage
StudyPlan	Personalized plans	Subject, topics, days, completion
Progress	Learning statistics	Activity, quiz performance, topic data

ER/Data Relationship Representation

USER

| | +---- LEARNING_HISTORY

|

+----- QUIZ_RESULT

|

+----- STUDY_PLAN

Progress is derived from the user's stored learning activity.

User-owned learning records are associated with the authenticated user.

6. Authentication and Security

Registration

- Full name, email, password and confirmation
- Input validation
- Duplicate-account prevention
- bcrypt password hashing

Login

- Email/password verification
- JWT token generation
- Protected API access
- Authenticated user identification

Security Rules

- Never expose GEMINI_API_KEY in frontend code.
- Never commit .env.
- Never store plain-text passwords.
- Validate incoming data.
- Protect authenticated endpoints.
- Return safe user-facing errors.
- Configure CORS appropriately.

Forgot Password

EduGenie includes the forgot-password interface and its loading, success, error and validation states. A production email-reset workflow should use secure, expiring reset tokens and an email provider when enabled.

7. Backend API Design

Method	Endpoint	Purpose
POST	/api/auth/register	Create account
POST	/api/auth/login	Authenticate student
GET	/api/auth/me	Get authenticated user
POST	/api/ai/tutor	AI Tutor
POST	/api/ai/explain	Topic explanation
POST	/api/ai/notes	Generate notes
POST	/api/ai/quiz	Generate quiz
POST	/api/ai/evaluate	Evaluate answer
POST	/api/ai/study-plan	Generate study plan
GET	/api/learning/history	Learning history
GET	/api/learning/progress	Progress data

Backend Responsibilities

- Authenticate requests
- Validate inputs
- Call Gemini service
- Persist learning data
- Return structured JSON
- Handle failures without exposing secrets

8. AI Tutor

The AI Tutor provides an interactive learning conversation where students can ask questions and receive explanations suited to their level.

Supported Interaction

- Concept explanations
- Follow-up questions
- Beginner/intermediate/advanced explanations
- Examples
- Step-by-step explanations
- Code explanations when appropriate

UI Behavior

- User messages are right-aligned indigo bubbles
- AI responses are left-aligned cards
- Responses support headings, lists and code blocks
- Copy/regenerate/feedback actions
- Suggestion chips
- Typing, loading, error and empty states
- Responsive mobile chat

9. Explain a Topic

The Explain a Topic feature converts difficult academic concepts into structured explanations.

Inputs

- Subject
- Topic
- Difficulty: Beginner / Intermediate / Advanced

Generated Output

- Simple Explanation
- Key Concepts
- Example
- Important Points
- Quick Summary

Actions

- Copy
- Regenerate
- Save
- Clear

10. Generate Notes

The Notes Generator creates structured study material from a subject and topic.

Inputs

- Subject
- Topic
- Additional instructions

Notes Structure

- Overview
- Key Concepts
- Important Definitions
- Examples
- Importance
- Quick Revision

Actions

- Edit
- Save
- Download
- Share
- Regenerate

The notes viewer uses a document-like layout for comfortable reading and revision.

11. Generate Quiz and Quiz Workflow

Quiz Configuration

- Subject
- Topic
- Difficulty: Easy / Medium / Hard
- Number of questions

Quiz Runner

- Actual generated question count is used everywhere
- Four options per question
- Selected option uses indigo styling
- Correct answers remain hidden until submission
- Timer and submission states
- Submit Quiz

Quiz Result

- Score percentage
- Correct
- Wrong
- Total
- Accuracy
- Solutions
- Retake
- Dashboard navigation

Example: a 10-question quiz displays 'Question 2 of 10' and 'Total 10'. Conflicting hardcoded totals are not used.

12. Evaluate Answer

Students can submit a question and their answer for AI-based educational feedback.

Inputs

- Question
- Your Answer

Feedback

- Score
- Correctness
- What was done well
- Missing points
- Correct explanation
- Suggestions for improvement

The feature is designed to explain learning gaps instead of providing only a numeric score.

13. Personalized Study Plan

Study Plan generates a realistic day-wise schedule based on the student's goals and available time.

Inputs

- Subject
- Topics
- Number of days
- Daily study time
- Difficulty / goal

Plan

- Day number
- Topic
- Learning objective
- Practice task
- Completion status

Actions

- Mark complete
- Regenerate
- Save plan

14. Dashboard and Progress Tracking

Dashboard

- Welcome message using authenticated user's name
- Quizzes Completed
- Average Score
- Topics Studied
- Study Streak
- Eight feature quick actions
- Recent activity
- Learning progress chart

Progress

- Overall progress
- Quiz performance
- Average score
- Topics studied
- Study streak
- Weekly activity
- Topic distribution

Statistics should come from backend/database data rather than permanently hardcoded sample values.

15. Learning History

Learning History provides a central record of previous learning activities.

Activity Types

- AI Tutor
- Explanation
- Notes
- Quiz
- Answer Evaluation
- Study Plan

Interface

- Activity type
- Topic
- Date/time
- Score where applicable
- Status
- Search
- Filters
- Pagination/load-more when required

An empty state is shown when the user has no learning activity.

16. Frontend UI/UX Design

Element	Specification
Primary	#4F46E5 / #5B4CF0
Sidebar	#0B0F3B
Page background	#F5F6FB
Success	#22C55E
Error	#EF4444
Heading	#1E293B
Muted	#64748B
Font	Inter or Poppins
Cards	16–20px radius, soft shadows
Style	Modern SaaS, professional EdTech

Responsive Behavior

- Desktop: full sidebar and multi-column dashboard
- Tablet: compact/collapsible navigation
- Mobile: drawer navigation and single-column layout
- Responsive charts/forms/chat/quiz
- No horizontal overflow

17. Profile, Settings and Forgot Password

Profile

- Avatar
- Full name
- Email
- Account information
- Learning statistics
- Edit Profile
- Change Password

Settings

- Account
- Appearance
- Notifications
- Privacy/data preferences

Forgot Password

- Email
- Validation
- Loading
- Success
- Error
- Reset-password screen when backend support is enabled

18. React Components and Routing

Reusable Components

- Sidebar
- Topbar
- StatCard
- ActionTile
- Button
- Input
- Select
- Modal
- ActivityList
- ProgressChart
- ChatBubble
- ChatInput
- SuggestionChips
- QuizQuestion
- OptionButton
- Timer
- ResultCard
- NoteView
- StudyDayCard
- LoadingSpinner
- EmptyState
- ErrorMessage
- ProtectedRoute

Routes

/

/login

/register
/forgot-password

/dashboard
/ai-tutor
/explain
/notes
/quiz
/quiz/:id
/quiz/:id/result
/evaluate
/study-plan
/history
/progress
/profile
/settings

Authenticated routes are protected so student-specific data is not accessible without login.

19. Testing, Verification and Local Execution

Testing Checklist

- Registration and login
- Protected routes
- Gemini requests
- Quiz generation/submission
- Answer evaluation
- Study plan
- MongoDB operations
- Learning history
- Progress
- Responsive UI
- Loading/empty/error/success states
- Build/runtime verification

Local Execution

Use the package.json scripts defined in the current project. Typical commands are:

```
cd backend
```

```
npm install
```

```
npm run dev
```

```
cd frontend
```

```
npm install
```

```
npm run dev
```

Repository and Secrets

- Git/GitHub version control
- Real .env kept private
- .env.example contains safe placeholders
- Gemini keys, MongoDB credentials and JWT secrets are never committed

20. Project Conclusion

EduGenie demonstrates how generative AI can be integrated into a structured educational workflow. It combines a modern React interface, secure Node.js/Express APIs, MongoDB Atlas persistence, JWT authentication and Google Gemini-powered learning services.

End-to-End User Journey

Landing Page

|

v

Register / Login

|

v

Dashboard

|

+----> AI Tutor

+----> Explain a Topic

+----> Generate Notes

+----> Generate Quiz ----> Quiz Result

+----> Evaluate Answer

+----> Study Plan

|

v

Learning History

|

v

Progress Tracking

Final Quality Goals

- Professional product-style learning experience
- Reusable React components
- Secure Gemini integration
- MongoDB persistence
- Responsive UI/UX

- Clear application states
- Extensible architecture

Team

Member	Role
Kamali S.	Frontend / AI Integration
Kavikuman. M	Backend Development
Ragul Gandhi. B	Database / Backend Support

Deployment Status: Not publicly deployed at the time of this documentation.

End of EduGenie Project Documentation.